

Week 4 - Python Fundamentals: Quiz Recap

Full solution and instructor cheatsheet

Aim: predict the output, its type, and whether an object changes. Use base Python only. All data here are synthetic examples.

Time	Topic	Main question
0-2 min	Object types and operations	What class does this object belong to?
2-8 min	Mutability and identity	Same value, or the same object?
8-13 min	Indexing and selecting	One item, a slice, or a dictionary key?
13-18 min	Conditions and ternary expressions	Which branch runs?
18-24 min	User-defined functions and loops	What is printed, returned, or changed?
24-25 min	Exit check	Can we explain the output before running?

20-minute route: omit the sorting cell and function-mutation cell; use one exit question. **30-minute route:** use the 20-minute route, then file discovery (7 min) and text I/O (3 min). The custom-class example (2 min) is an extra or a swap for another example.

Teaching rhythm: ask for a prediction before each cell, run it, then explain the result. The paired week4_fundamentals_empty.ipynb has the same demo order with comment-only code cells. Restart the kernel and run all cells to reset the examples. Run mutation cells again only after rerunning their setup; otherwise state can accumulate.

Topic 0 - Object Types and Basic Operations (2 min)

`type(obj)` gives the object's class. A value such as 3 is an instance of the built-in class `int`. Lists and dictionaries are mutable; numbers, Booleans, strings and tuples are immutable. A tuple can contain a mutable object: its slots cannot be reassigned, but a contained list can change. Dictionary keys preserve insertion order in modern Python; select values by key.

CODE

```
examples = [True, 3, 2.5, "Zurich", [1, 2], (1, 2), {"city": "Zurich"}]
for obj in examples:
    print(type(obj))

mixed = [str(8), 1, {"name": "Demo"}]
print(type(mixed), type(mixed[0]))
print(100 + int("250"), "100" + "250")
print(7 / 2, 7 // 2, 7 % 2, 2 ** 3)
```

OUTPUT

```
<class 'bool'>
<class 'int'>
<class 'float'>
<class 'str'>
<class 'list'>
<class 'tuple'>
<class 'dict'>
<class 'list'> <class 'str'>
350 100250
3.5 3 1 8
```

Say aloud: a list containing a string is still a list. `+` adds numbers or joins strings; `100 + "250"` raises `TypeError`. `/` gives a float; `//` is floor division, `%` is remainder, and `**` is exponentiation. Names are case-sensitive: `MY_STRING` is a different name from `my_string`. A name that has never been assigned raises `NameError`.

Topic 1 - Mutability, Classes and Object IDs (6 min)

A variable is a name bound to an object. Assignment binds another name; it does not copy a list. `==` compares values, `is` compares identity, and `id()` identifies an object during its lifetime. The numeric ID is session-specific and may be reused after an object is gone. Use `==` for numbers and strings; use `is None` to check for `None`.

Predict: after appending through one name, which lists change?

CODE

```
cities = ["zurich", "geneva"]
alias = cities
copied = cities.copy()

print(cities == copied, cities is copied)
print(cities is alias, id(cities) == id(alias))
before_id = id(cities)
result = alias.append("bern")
print(cities, alias, copied)
print(id(cities) == before_id)
print(result, type(result))
```

OUTPUT

```
True False
True True
['zurich', 'geneva', 'bern'] ['zurich', 'geneva', 'bern'] ['zurich', 'geneva']
True
None <class 'NoneType'>
```

Answer: the alias and original show Bern; the copy does not. The list keeps its ID when mutated. `.append()` changes the list and returns `None`. Avoid `cities = cities.append("bern")`: it would bind `cities` to `None`. `.copy()` and a list slice make a **shallow** copy: nested mutable elements remain shared.

CODE

```
prices = [12, 3, 8]
ordered = sorted(prices)
print(prices, ordered)
result = prices.sort()
print(prices, result)
```

OUTPUT

```
[12, 3, 8] [3, 8, 12]
[3, 8, 12] None
```

CODE

```
city = "zurich"
old_city = city
city = city.upper()
print(old_city, city)
print(type(city))
```

OUTPUT

```
zurich ZURICH
<class 'str'>
```

Say aloud: `sorted()` creates a new list; `.sort()` mutates and returns `None`. String methods return strings; they do not mutate the original string. Rebinding a name changes what that name refers to. A class describes a kind of object; `type(city)` returns `str`. A short user-defined class is available in the optional extension.

Topic 2 - Indexing, Slicing and Selecting (5 min)

Sequence positions start at 0; -1 is the last item. Slices are `seq[start:stop:step]`: the stop is excluded. A single index gives an element; a slice gives a new sequence of the same type for strings, lists and tuples. An out-of-range index raises `IndexError`; an oversized slice is allowed.

Predict: the value and type of `fruits[1]`, `fruits[1:2]` and `fruits[2][:3]`.

CODE

```
word = "university"
print(word[1:4], word[-1], word[::-1])
fruits = ["apple", "orange", "mango"]
print(fruits[1], type(fruits[1]))
print(fruits[1:2], type(fruits[1:2]))
print(fruits[2][:3], fruits[-2:])
print(fruits[:99])
```

OUTPUT

```
niv y ytisrevinu
orange <class 'str'>
['orange'] <class 'list'>
man ['orange', 'mango']
['apple', 'orange', 'mango']
```

CODE

```
record = {"city": "Zurich", "scores": [4, 5, 6]}
print(record["city"], record["scores"][1])
print(record.get("country", "unknown"))

for fruit in fruits:
    print(fruit[1], fruits[1])
```

OUTPUT

```
Zurich 5
unknown
p orange
r orange
a orange
```

Answers: `fruits[1]` is a string, `fruits[1:2]` is a list, and the nested slice is "man". `record["scores"][1]` is 5. A missing key with `record["country"]` raises `KeyError`; `.get()` supplies a fallback. Inside the loop, `fruit` changes each iteration; `fruits[1]` keeps selecting "orange".

Topic 3 - Conditionals and Ternary Expressions (5 min)

Comparisons give Booleans. Combine conditions with and, or and not. if / elif / else runs the first matching branch. Independent if statements can each run. Empty strings and containers, zero, and None are falsy; non-empty strings such as "False" are truthy.

CODE

```
x, y, z = 10, 15, 20
if y >= z or x >= y:
    print("hello")
elif z > x:
    print("goodbye")
else:
    print("see you")

print(10 == 10.0, 10 > 10.0)
```

OUTPUT

```
goodbye
True False
```

CODE

```
name = ""
score = 5
if score >= 4:
    print("pass")
if score >= 4 and name:
    print("named result")
if not name:
    print("missing name")

label = "pass" if score >= 4 else "retry"
print(label, bool(name), bool("False"))
```

OUTPUT

```
pass
missing name
pass False True
```

Say aloud: a ternary expression is value_if_true if condition else value_if_false. It produces a value that can be assigned or returned. For score = 4, the label is "pass"; for score = 3, it is "retry". Only the chosen expression is evaluated. = assigns; == compares. and / or short-circuit and return an operand; they do not always return a Boolean (True and "" gives "").

Topic 4 - User-Defined Functions and Loops (6 min)

`def` defines a function; a call runs its body. Parameters are local names bound to supplied arguments. `return` hands a value back to the caller and ends the call. `print()` displays a value and returns `None`. A function that reaches the end without `return` also returns `None`.

CODE

```
def addition(a, b):
    print(a + b)

result = addition(10, 5)
print(result)

def division(x, y):
    return x / y

print(division(5, 10))
```

OUTPUT

```
15
None
0.5
```

CODE

```
def increment(number, by=1):
    return number + by

x = 0
for i in range(5):
    print(increment(x) if i < 2 else increment(x, by=2))
    print("x:", x)

for i in range(3):
    x = increment(x)
    print("updated x:", x)
```

OUTPUT

```
1
1
2
2
2
x: 0
updated x: 3
```

Answers: `addition` prints 15, then the caller prints `None`; `division` returns 0.5. The first `increment` loop prints 1, 1, 2, 2, 2 on separate lines, then `x: 0`. The second loop assigns each return value, so the final `x` is 3. `range(5)` yields 0, 1, 2, 3, 4; its stop is excluded.

CODE

```
def add_city(items, city):
    items.append(city)

def reset_cities(items):
    items = []
    return items

route = ["zurich"]
print(add_city(route, "bern"))
reset_route = reset_cities(route)
print(route, reset_route)
```

OUTPUT

```
None
['zurich', 'bern'] []
```

Say aloud: the caller and parameter refer to the same list, so appending is visible to the caller. Assigning `items = []` rebinds the local name and leaves the caller's list unchanged. `add_city` returns `None`; `reset_cities` returns a new empty list. Prefer immutable defaults such as `by=1`; a mutable default like `items=[]` is shared across calls.

Exit Check (1 min)

Ask without running code:

1. Is `fruits[1:2]` a string or a list? **A list.**
2. Does a function call automatically update its numeric argument? **No; assign the return.**
3. What does a function with only `print(...)` return? **None.**

Quiz reading checklist: track names and types; mark slice stops; trace branches in order; trace every loop iteration; distinguish printed output from returned values; check mutation versus rebinding.

Optional Extension - A Small Custom Class (2 min)

Use as an extra or swap. `class` defines a new object type; calling the class creates an instance. `__init__` initializes the instance; `self` refers to that instance. An attribute stores data on an object. This class allows its attributes to change, so its instances are mutable.

CODE

```
class City:
    def __init__(self, name):
        self.name = name

    def rename(self, name):
        self.name = name

demo_city = City("Zurich")
same_city = demo_city
before_id = id(demo_city)
demo_city.rename("Bern")
print(type(demo_city).__name__)
print(same_city.name, same_city is demo_city)
print(id(demo_city) == before_id)
```

OUTPUT

```
City
Bern True
True
```

File I/O - Finding Files and Folders (7 min)

First **list paths**, inspect the selection, then **read file contents**. Path represents a file or folder path; use `/` to join path components. `iterdir()` lists immediate contents; `rglob("**")` searches recursively through subfolders. Use `.is_file()` or `.is_dir()` to keep the kind you want, and `sorted()` for predictable display order.

The synthetic files already exist in `week_4/data_raw/file_io_demo`; open this folder in Finder before running the examples. `Path.cwd()` shows the kernel's working directory. The setup supports starting in the repository root, `week_4`, or `week_4/scripts`, and constructs the same absolute demo path in each case.

These examples search only the synthetic demo folder. For another exercise, choose a narrow, approved course-data folder. Cryptomator Vaults, including decrypted mounts, are human-only and must never be accessed. These examples demonstrate selection; they are not filesystem access-control enforcement.

Demo tree: the root has `README.txt` and `overview.csv`; `data/` has `cities.csv`, `notes.txt` and `archive/old_cities.csv`; `reports/` has `summary.txt`.

CODE

```
from pathlib import Path

working_folder = Path.cwd()
if working_folder.name == "scripts":
    week_4_folder = working_folder.parent
elif working_folder.name == "week_4":
    week_4_folder = working_folder
else:
    week_4_folder = working_folder / "sessions" / "week_4"

root = week_4_folder / "data_raw" / "file_io_demo"
print("Working directory:", working_folder)
print("Demo folder:", root)
print("Folder exists:", root.is_dir())
```

OUTPUT

```
Working directory: /Users/justinasgrigaitis/Documents/Github/ba-programming-2026/sessions/week_4/scripts
Demo folder:
/Users/justinasgrigaitis/Documents/Github/ba-programming-2026/sessions/week_4/data_raw/file_io_demo
Folder exists: True
```

CODE

```
files_here = sorted(p for p in root.iterdir() if p.is_file() and not p.name.startswith("."))
folders_here = sorted(p for p in root.iterdir() if p.is_dir() and not p.name.startswith("."))

for path in files_here:
    print("file:", path.relative_to(root))
for path in folders_here:
    print("folder:", path.relative_to(root))
```

OUTPUT

```
file: README.txt
file: overview.csv
folder: data
folder: reports
```

CODE

```
all_files = sorted(p for p in root.rglob("**") if p.is_file() and not p.name.startswith("."))
all_folders = sorted(p for p in root.rglob("**") if p.is_dir() and not p.name.startswith("."))

for path in all_files:
    print("file:", path.relative_to(root))
for path in all_folders:
    print("folder:", path.relative_to(root))
```

OUTPUT

```
file: README.txt
file: data/archive/old_cities.csv
file: data/cities.csv
file: data/notes.txt
file: overview.csv
file: reports/summary.txt
folder: data
folder: data/archive
folder: reports
```

Inspect: the immediate list has **2 files and 2 folders**; the recursive list has **6 files and 3 folders**. The root itself is not included. `relative_to(root)` removes the shared directory prefix so nested locations are easy to see. Listings skip hidden files such as `.DS_Store` and do not read file contents.

File I/O - Selecting and Inspecting a Subset

`glob("*.csv")` selects CSV paths in the chosen directory; `rglob("*.csv")` includes subfolders. `*` matches a filename pattern. Start from `root / "data"` to limit the search to that subpath. Keep `.is_file()` because a folder can also have a name ending in `.csv`.

Predict: how many CSV files will each selection contain?

CODE

```
csv_here = sorted(p for p in root.glob("*.csv") if p.is_file())
csv_anywhere = sorted(p for p in root.rglob("*.csv") if p.is_file())

print("CSV in root:", len(csv_here))
for path in csv_here:
    print(path.relative_to(root))

print("CSV recursively:", len(csv_anywhere))
for path in csv_anywhere:
    print(path.relative_to(root))
```

OUTPUT

```
CSV in root: 1
overview.csv
CSV recursively: 3
data/archive/old_cities.csv
data/cities.csv
overview.csv
```

CODE

```
data_folder = root / "data"
data_csv = sorted(p for p in data_folder.rglob("*.csv") if p.is_file())

print("CSV below data/:", len(data_csv))
for path in data_csv:
    print("Relative path:", path.relative_to(root))
    print("Absolute path:", path)
    print(path.read_text(encoding="utf-8").strip())
```

OUTPUT

```
CSV below data/: 2
Relative path: data/archive/old_cities.csv
Absolute path: /Users/justinasgrigaitis/Documents/Github/ba-programming-2026/sessions/week_4/data_raw/file_
io_demo/data/archive/old_cities.csv
city
Bern
Relative path: data/cities.csv
Absolute path: /Users/justinasgrigaitis/Documents/Github/ba-programming-2026/sessions/week_4/data_raw/file_
io_demo/data/cities.csv
city
Zurich
```

Answers: root-only CSV selection has **1** file; recursive selection has **3**; the `data/` subset has **2**, including its archive. Use `data_folder.glob("*.csv")` to exclude the archive; that leaves just `data/cities.csv`. Check the printed paths as well as the count: an archive may match the file type but be unwanted. `read_text()` reads a whole text file as a string; it does not parse CSV rows into a table. `relative_to(root)` produces a relative path; the original path includes the full location. The demo files remain on disk, so you can revisit them in Finder and rerun these cells.

File I/O - Writing and Reading Text (3 min)

Connect functions, loops and string methods to the earlier file I/O example. Clean synthetic city names, write them to `week_4/output/file_io_demo/cities_demo.txt`, then read them back. `mkdir(parents=True, exist_ok=True)` creates the output folder if needed; the file remains visible in Finder. Keep output separate from the demo inputs so the earlier file-count examples do not change. Run the file-discovery setup first to define `week_4_folder`.

CODE

```
from pathlib import Path

def clean_city(text):
    return text.strip().title()

raw_lines = [" zurich ", "", " GENEVA", "bern "]
cleaned = []
for line in raw_lines:
    if line.strip():
        cleaned.append(clean_city(line))

output_folder = week_4_folder / "output" / "file_io_demo"
output_folder.mkdir(parents=True, exist_ok=True)
path = output_folder / "cities_demo.txt"
path.write_text("\n".join(cleaned), encoding="utf-8")
print("Written file:", path)
print(path.read_text(encoding="utf-8"))
```

OUTPUT

```
Written file: /Users/justinasgrigaitis/Documents/Github/ba-programming-2026/sessions/week_4/output/file_io_
demo/cities_demo.txt
Zurich
Geneva
Bern
```

Say aloud: `.strip()` removes surrounding whitespace; `.title()` returns a new string. `"\n".join(cleaned)` joins names with newline separators. `write_text()` creates a file or replaces its contents; `read_text()` returns those contents as a string. The output persists between runs. Open `output/file_io_demo/cities_demo.txt` in Finder to compare it with the printed result.

Reference basis: supplied Week 1-2 notebooks, the 2025 quiz PDF and its class-edit script. The optional class and I/O examples extend the historical quiz scope.